Agentic network for requirements refinement

Week 04, Task 1. Juraj Petras

Introduction

The input is 20 plain-text files about the reports module of PINES, a library consortium running Evergreen, all from 2009. They are an early SRS draft, meeting minutes, two interviews, e-mail threads, a staff survey and a reference description of the old system.

The network is a LangGraph graph in `src/req_refinery/` with two versions. I built `network` first. It extracts candidates per document, merges duplicates by their text in `merge_dedup` and checks coverage per document, so no call ever sees the whole corpus. A review loop then refines the list for up to 5 rounds. Its best run before the hybrid got 6.5/10 from Claude Opus 5.5, which graded all the documents.

I then compared it with a baseline, `src/req_refinery/baseline.py`, that sends all 20 documents to `gpt-5.6-luna` in one call with the output criteria, the requirement definition, the priority scale and a document template. It got 6/10. It missed many items, but it had far fewer duplicates than the network, because one call could compare everything at once.

The `hybrid` version combines the two. It keeps the per-document `extract` as a checklist and replaces `merge_dedup` and the coverage pass with one `consolidate` call that, like the baseline, sees all 20 documents. Its document, `output/hybrid/requirements.md`, has 136 requirements, 6 blacklisted items and 6 stakeholder questions, and Claude Opus 5.5 rated it 7.5/10.

The network

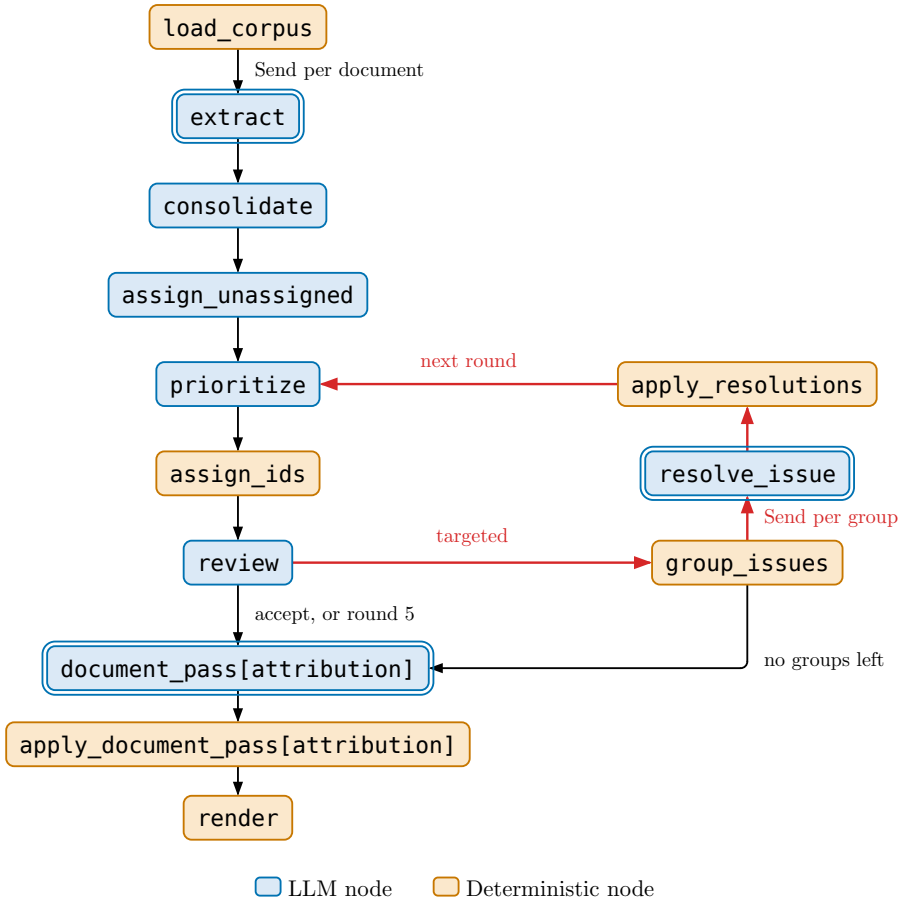


Figure 1: The `hybrid` graph, the default. Red edges are the review loop, which runs when `review` returns the route `targeted`. Nodes with a double border run once per document or once per issue group, in parallel through LangGraph `Send`.

Node	Type	Model	What it does
`load_corpus`	code	–	Reads every text file under the input dir and numbers them D01 to Dn.
`extract`	LLM	`gpt-5.6-luna`	One call per document. Returns the candidate requirements of that document.
`consolidate`	LLM	`gpt-6-luna`	One call with all documents in full and all candidates as a checklist. Returns requirements with sources, a glossary, a blacklist and stakeholder questions.
`merge_dedup`	LLM	`gpt-5.6-luna`	Only in `network`. Merges duplicate candidates from their text alone.
`assign_unassigned`	LLM	`gpt-5-mini`	`consolidate` and `merge_dedup` should fold each candidate into a requirement or into the blacklist, but they can skip a few. This node gets the skipped ones. If an existing requirement already says the same thing, it adds the candidate's document to that requirement's sources. Otherwise it adds the candidate as a new requirement. In the final hybrid run 4 of 327 candidates were skipped, 1 was mapped and 3 were added.
`document_pass[coverage]`	LLM	`gpt-5.6-luna`	Only in `network`. Per document, finds which requirements it supports and what is missing.
`prioritize`	LLM	`gpt-6-luna`	Gives High, Medium or Low with a one-sentence reason to items without a priority.
`assign_ids`	code	–	Sorts by priority, source and name, then numbers R001 to Rn.
`review`	LLM	`gpt-6-luna`	Checks the whole list and returns typed issues and the route.
`group_issues`	code	–	Joins issues that share a requirement into groups and drops granularity issues on settled items.
`resolve_issue`	LLM	`gpt-5.6-luna`	One call per group. Returns replacement requirements and blacklist entries.
`apply_resolutions`	code	–	Deletes the involved requirements, inserts the replacements and validates the list.
`document_pass[attribution]`	LLM	`gpt-5.6-luna`	Runs after the loop. Per document, finds which requirements it supports.
`apply_document_pass[attribution]`	code	–	Rewrites the sources of every requirement. Statements that no requirement covers become `possibly missed` open issues.
`render`	code	–	Writes `requirements.md` and `requirements.json`.

How it works

Shared state. All nodes read and write one `GraphState`, a `TypedDict` with the documents, candidates, requirements, glossary, blacklist, issues and round number. Parallel calls append to their lists through a reducer. A node that wants to overwrite a list wraps it in `Replace`. Every requirement is a Pydantic model, and the code validates it again after each change. The check also confirms that its sources are real document IDs.

Fan-outs. LangGraph `Send` starts `extract`, `resolve_issue` and both `document_pass` modes, one call per document or per issue group. LangGraph waits for all of them before the next node, so the next deterministic node sees all results at once. At most 8 API calls run at the same time.

The review loop. `review` runs on `gpt-6-luna`. It sees the whole list with glossary and blacklist and returns issues of seven types: `duplicate`, `conflict`, `wording`, `nonsense`, `priority`, `too_broad` and `too_narrow`. With the route `targeted` the issues go to `group_issues`. That node joins issues that share a requirement into one group, so two issues about the same requirement never produce two rewrites of it. Each group gets one `resolve_issue` call. `apply_resolutions` then deletes every involved requirement and inserts the replacements. This delete-and-insert step is the only way the loop changes the list. Replacements have no priority yet, so the loop goes back through `prioritize` and `assign_ids` before the next review.

IDs. `assign_ids` numbers the whole list again on every pass. The loop splits and merges items all the time, and a split item has no obvious old ID to keep, so IDs are plain sequence numbers. The resolver has to refer to other requirements by name, and `apply_resolutions` replaces any leftover `R` number in its text with the requirement's name.

Origin tags. Every requirement records where it came from, for example `consolidate`, `unassigned`, `split` or `folded`. In early runs the reviewer split an item in one round and asked to merge the pieces again in the next. Now the reviewer sees items from a split or a fold with the tag `[granularity settled]`, and `group_issues` drops any `too_broad` or `too_narrow` issue on them. The other issue types still apply.

Round cap. The loop runs at most 5 rounds. If the reviewer still says `targeted` in round 5, the code forces the route to `accept` and writes the remaining issues to the open issues section of the document.

Attribution pass. After the loop, `document_pass[attribution]` reads every document again against the final list and returns the requirements that document supports. The deterministic step then sets the sources of every requirement to exactly those documents. Anything a document states that no requirement covers becomes a `possibly missed` open issue. In the final hybrid run this step added 58 source citations and removed 32.

Blacklist. A blacklist entry holds the full discarded requirement and a one-paragraph reason that names the sources on each side. Entries come from `consolidate`, for nonsensical or conflicting candidates, and from `resolve_issue`, for `conflict` and `nonsense` issues. The document lists them as B01 to Bn after the requirements.

The `network` variant

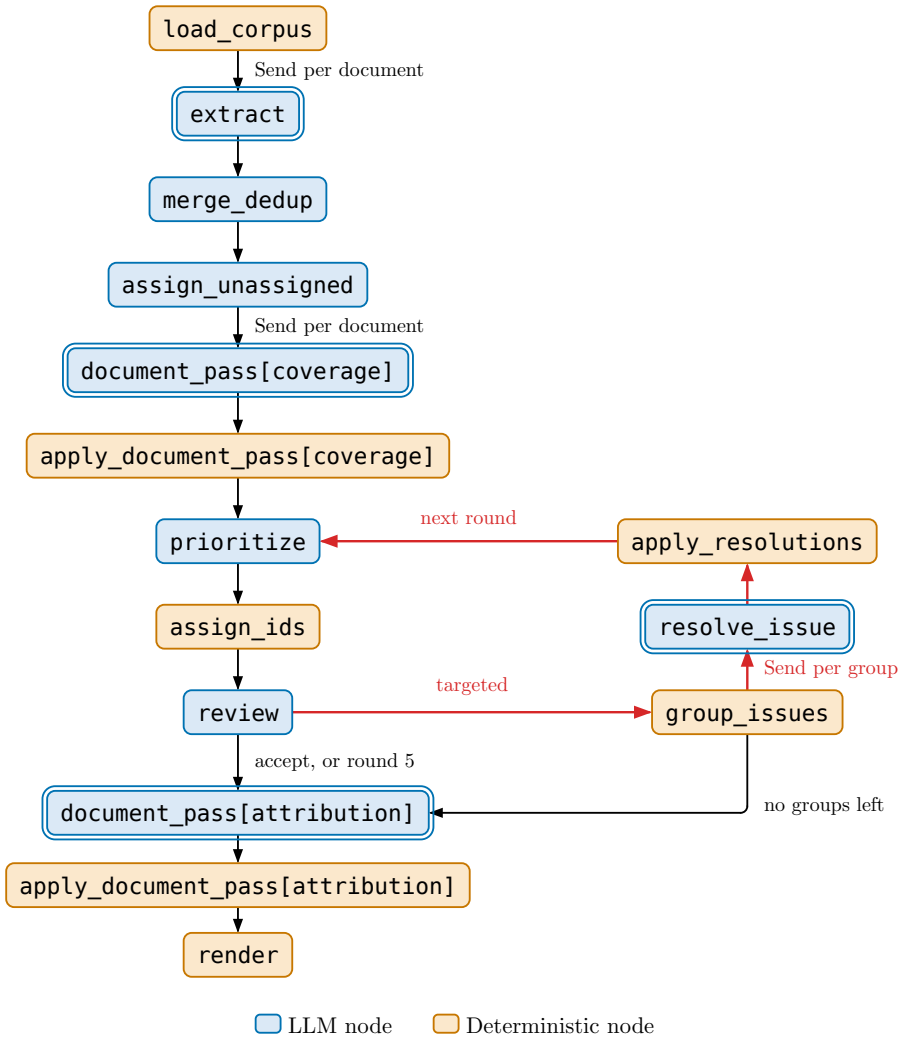


Figure 2: The `network` graph, selected with `--graph network`. From `prioritize` on it has the same review loop and tail as Figure 1.

The `network` graph was my first design. No call in it sees the whole corpus. `merge_dedup` merges the candidates from their text alone, and a per-document `document_pass[coverage]` then finds what each document supports and what is still missing. The `hybrid` graph replaces those two nodes with one `consolidate` call that gets all 20 documents in full plus every candidate. With the full text it judges duplicates, sources and conflicts better, and it is the only node that writes stakeholder questions. That one call used 75,076 tokens. For a corpus over about 300k tokens the node stops with an error that says to use `--graph network`.

Models used

Node	Model	Why
`extract`	`gpt-5.6-luna`	20 parallel calls that have to follow the granularity rules. On `gpt-5-mini` the items were too fine-grained.
`consolidate`	`gpt-6-luna`	The hardest judgement, made once, so it gets the strongest model.
`assign_unassigned`	`gpt-5-mini`	Maps a few leftover items onto a numbered list.
`document_pass`	`gpt-5.6-luna`	Reads one long document against the whole list, 20 calls per pass.
`prioritize`	`gpt-6-luna`	On `gpt-5.4-nano` items that a source calls critical ended up at Low.
`review`	`gpt-6-luna`	Judges the whole list at once. The loop is only as good as this call.
`resolve_issue`	`gpt-5.6-luna`	Many small rewrite calls.

Model	Tokens, hybrid run	Tokens, network run
`gpt-6-luna`	250,215	174,060
`gpt-5.6-luna`	366,634	638,326
`gpt-5-mini`	7,636	9,429
Total	**624,485**	**821,815**

The network uses three models, `gpt-6-luna`, `gpt-5.6-luna` and `gpt-5-mini`. At first `prioritize` ran on `gpt-4.1-nano`. In `20261024_162519` that model returned 404 because it was deprecated, so I switched to `gpt-5.4-nano`. In `20261024_172523` I moved `prioritize` to `gpt-6-luna` and `extract` from `gpt-5-mini` to `gpt-5.6-luna`, because those two nodes produced the weakest parts of the output.

Trace and timing

Every run writes a folder `output/runs/<timestamp>_<graph>/` with these files.

- `trace.jsonl` has one record per node execution, LLM or deterministic, with node name, round, start, end and duration. LLM records also hold every call with the messages sent, the raw response, the parsed output, the token usage and any retry.
- `state/NN_<node>.json` has the full graph state after each LangGraph step.
- `requirements.md` and `requirements.json` are the document of that run.

The two timelines below come straight from the traces of the final runs. Each node has a row and each execution a bar. Parallel calls stack as thin bars in the same row. Their bars include the time a call waits for one of the 8 API slots, so for them the span of the whole row is the useful number.

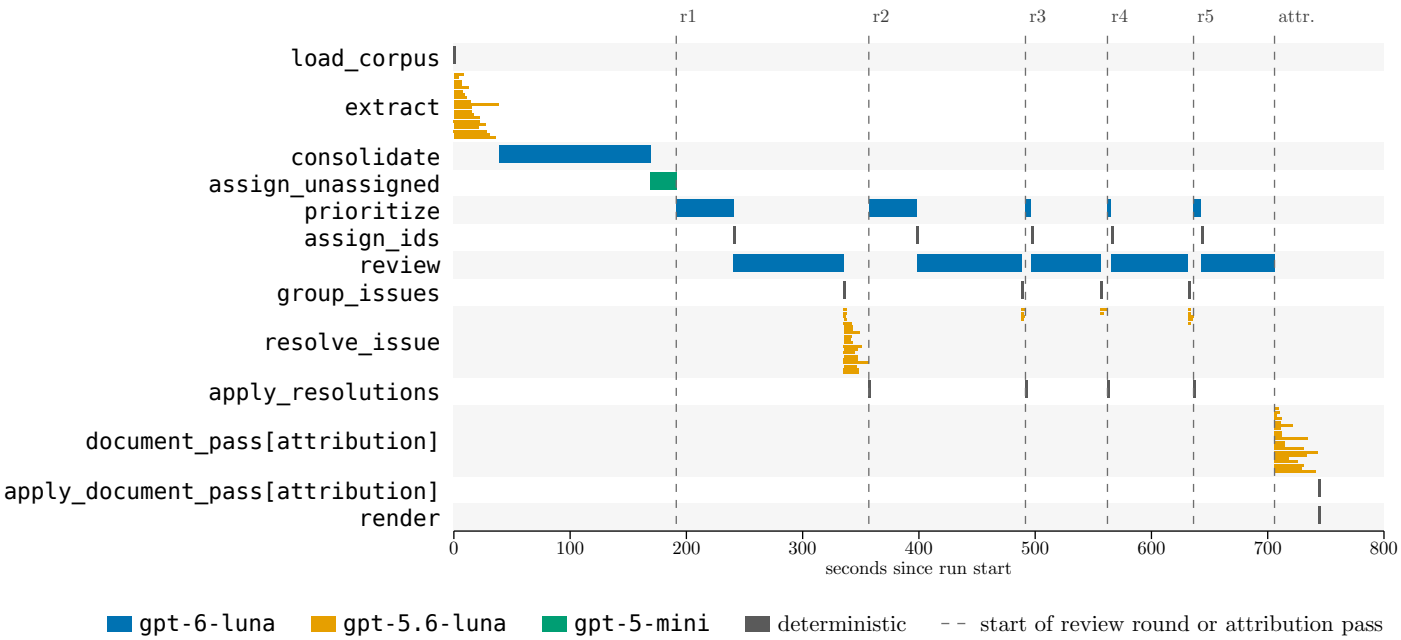


Figure 3: Timeline of the final `hybrid` run, 744 s wall time. The lines r1 to r5 mark the start of each review round at `prioritize`, and attr. marks the start of the attribution pass.

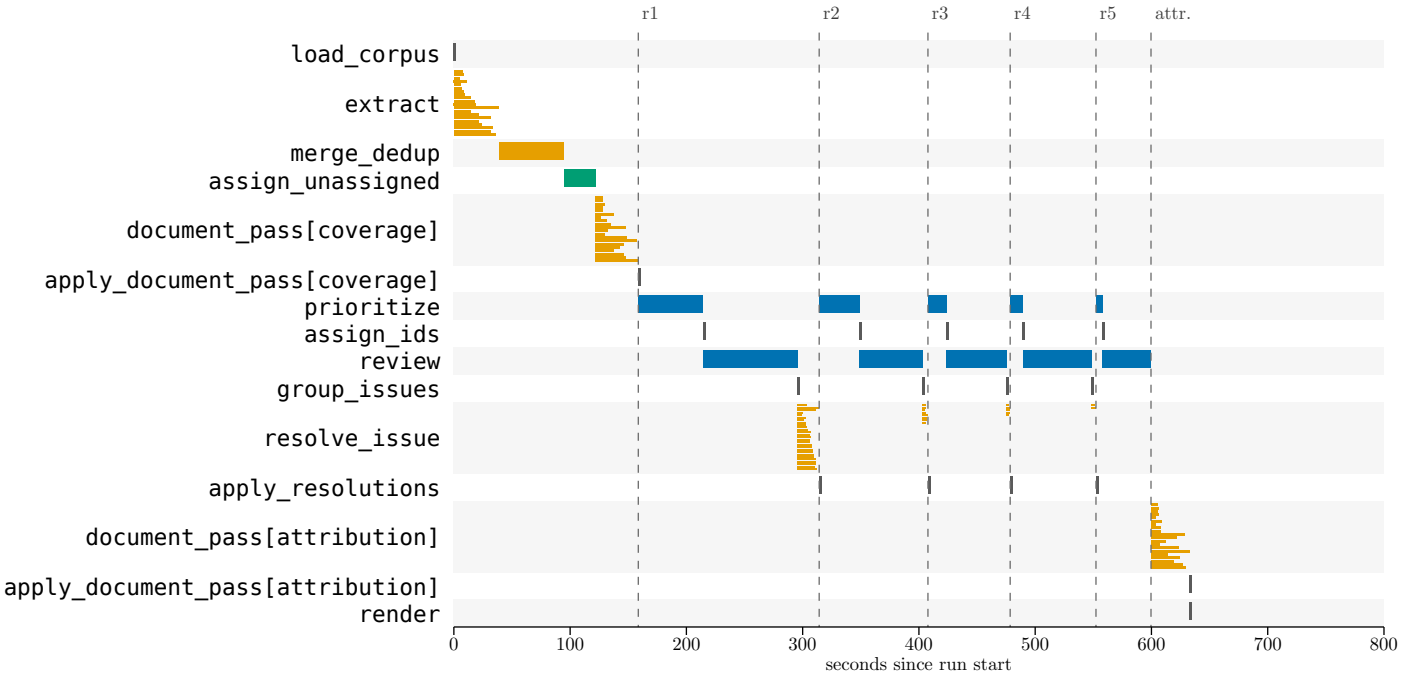


Figure 4: Timeline of the final `network` run, 633 s wall time, with the same time axis and colours as Figure 3.

	hybrid				network			
Node	**Calls**	**Time**	**Share**	**Tokens**	**Calls**	**Time**	**Share**	**Tokens**
`load_corpus`	1	< 0.1 s	0%	–	1	< 0.1 s	0%	–
`extract`	20	39 s	5%	78k	20	39 s	6%	78k
`consolidate`	1	130 s	18%	75k				
`assign_unassigned`	1	22 s	3%	8k	1	27 s	4%	9k
`prioritize`	5	105 s	14%	56k	5	123 s	19%	59k
`assign_ids`	5	< 0.1 s	0%	–	5	< 0.1 s	0%	–
`review`	5	374 s	50%	119k	5	287 s	45%	115k
`group_issues`	4	< 0.1 s	0%	–	4	< 0.1 s	0%	–
`resolve_issue`	31	35 s	5%	63k	42	31 s	5%	93k
`apply_resolutions`	4	< 0.1 s	0%	–	4	< 0.1 s	0%	–
`document_pass[attribution]`	20	38 s	5%	226k	20	33 s	5%	218k
`apply_document_pass[attribution]`	1	< 0.1 s	0%	–	1	< 0.1 s	0%	–
`render`	1	< 0.1 s	0%	–	1	< 0.1 s	0%	–
`merge_dedup`					1	56 s	9%	31k
`document_pass[coverage]`					20	37 s	6%	218k
`apply_document_pass[coverage]`					1	< 0.1 s	0%	–

Calls counts node executions. *Time* is the wall time during which at least one call of the node was running. *Share* is that time over the wall time of the run.

- `review` is the slow part. Its 5 calls take 50% of the hybrid run, about 75 s each. The loop is sequential, so five rounds mean five of these waits.
- The fan-outs take little time. 20 `extract` calls finish in 39 s. The attribution pass uses the most tokens, 226k, because each of its 20 calls gets the whole requirement list.
- All deterministic nodes of the hybrid run take 18 ms together.

Spec coverage

Spec	What addresses it	Result in the hybrid document
ANS1	LLM nodes `extract`, `consolidate`, `assign_unassigned`, `prioritize`, `review`, `resolve_issue`, `document_pass[attribution]`.	7 LLM nodes in the hybrid graph.

Spec	What addresses it	Result in the hybrid document
ANS2	Deterministic nodes `load_corpus`, `assign_ids`, `group_issues`, `apply_resolutions`, `apply_document_pass[attribution]`, `render`.	6 deterministic nodes.
ANS3	The review loop runs `review`, `group_issues`, `resolve_issue`, `apply_resolutions`, `prioritize`, `assign_ids` and then `review` again. Every structured call is also retried once with its validation error.	Used in every run. The final run used all 5 rounds, and the loop rarely ends on its own.
ANS4	`gpt-6-luna`, `gpt-5.6-luna` and `gpt-5-mini`.	Met.
ANS5	LangGraph `StateGraph` with `Send` fan-outs and conditional edges.	Met.
OS1	`render` writes one `requirements.md` and the same content as JSON.	One document with scale, glossary, requirements, blacklist, open issues and a coverage table.
OS2	`assign_ids` numbers R001 to Rn, and the blacklist uses B01 to Bn.	Unique within the document. The numbers change between runs, and the document says so.
OS3	`consolidate`, the mapping in `assign_unassigned`, the review issue type `duplicate` and the resolver merge.	Partly met. Opus 5.5 still counted about 20 items that need merging, and 2 duplicate pairs are open issues.
OS4	Source document IDs on every requirement, set again per document by the attribution pass, and a per-document coverage table.	1 of 25 sampled requirements had a wrong source.
OS5	A prompt rule asks for a name of 3 to 8 words and a one-paragraph description. The schema rejects empty text.	Every requirement has both. Some descriptions are a single sentence.
OS6	`prioritize` with the scale from `config.py`, which the document prints at the top. Explicit priority markers in the sources force High.	21 High, 90 Medium and 25 Low, each with a reason.
OS7	The glossary from `consolidate`, the review issue types `wording` and `conflict`, and conflicts moved to the blacklist.	Wording is mostly consistent. The text uses "library system", which the glossary lacks, and 2 blacklisted items also appear as possibly missed.
OS8	Per-document `extract`, the candidate checklist in `consolidate`, `assign_unassigned` and the `possibly missed` issues of the attribution pass.	0 of the 12 items Opus 5.5 checked are missing. About 4 of the 9 `possibly missed` items are real. Scheduled purge and deletion reasons are missing, and no `possibly missed` issue flags them.
OS9	A blacklist section with the full item and a reason.	6 entries. B01 on batch efficiency and B06 on rapid distribution are questionable, and B04 and B05 are the same item.

How the design evolved

I ran the full graph 20 times while building it. The table lists every run from `run_history.csv`. "Own" means the reviewer accepted before the round cap. The grades come from Opus 5.5, as described under results, and "–" marks a run it did not grade.

Run	Graph	Change	Reqs	Blacklist	Open	Rounds	Own	Time	Tokens	Grade
20261022_233515	network	restructure node	36	1	5	3	no	292 s	342k	–
20261023_105641	network	connection error	failed	–	–	0		360 s	110k	–
20261023_110956	network	connection error	failed	–	–	1		441 s	289k	–
20261023_111902	network	granularity explosion	322	10	32	5	no	629 s	790k	–
20261023_114626	network	group_issues	250	10	14	5	no	680 s	669k	–
20261023_121042	network	assign_unassigned	272	14	5	5	no	684 s	729k	–
20261023_122842	network	prompt fixes	171	10	5	5	no	671 s	613k	–
20261024_162519	network	nano model 404	failed	–	–	0		283 s	271k	–
20261024_163032	network	gpt-5.4-nano	166	3	7	5	no	668 s	571k	–
20261024_165734	network	granularity guard	131	3	8	5	no	661 s	531k	5
20261024_172523	network	model swap	154	1	1	5	no	604 s	435k	6
20261024_174514	network	verify_sources	150	3	5	5	no	784 s	1619k	6.5
20261024_183024	network	document_pass	136	0	4	5	no	555 s	728k	6.5
20261024_184705_hybrid	hybrid	first hybrid	135	0	9	5	no	574 s	558k	7
20261024_191559_hybrid	hybrid	pointer ban	135	0	9	5	no	569 s	653k	7.5
20261024_194017_hybrid	hybrid	priority notes	132	0	7	4	yes	590 s	593k	7.5
20261024_195350_hybrid	hybrid	prompt tidy-up	120	4	20	5	no	558 s	567k	–
20261024_200309_network	network	network re-run	125	0	15	5	no	652 s	783k	–
20261024_202345_hybrid	hybrid	**final hybrid**	136	6	16	5	no	744 s	624k	7.5
20261024_203609_network	network	final network	126	0	10	5	no	633 s	822k	7
baseline	single call	one LLM call	51	2	5	–	–	61 s	43k	6

The changes that mattered:

- **20261022_233515.** The first design had a `restructure` route, where the reviewer could ask for the whole list to be rewritten in one call. That call returned a shorter list and dropped items without saying so. The run ended with 36 requirements. I removed the route. Since then every change goes through delete-and-insert on named requirements, and the trace shows each removed item.
- **20261023_111902.** This run ended with 322 requirements. The reviewer kept calling items `too_broad`, and the resolver split them into smaller and smaller pieces. Some issues also overlapped, so two resolver calls rewrote the same requirement and duplicated its content. I added `group_issues` for that, and one shared definition of a requirement, sized to one acceptance test, that every prompt includes.
- **20261023_121042.** At first the candidates that `merge_dedup` did not use were appended to the list, which created duplicates. `assign_unassigned` now maps each one onto an existing requirement when one covers it. The same run added origin tags.
- **Trace investigation on 20261023_122842.** I read the trace and the state snapshots round by round. Most granularity issues after round 1 were about items that an earlier resolution had just produced, so the loop was undoing its own splits and merges. The fine-grained items themselves came from `extract`, before the loop started. I fixed the extract prompt and added the granularity guard (the `[granularity settled]` tag) in `20261024_165734`.
- **Model swap in 20261024_172523.** Moving `extract` and `prioritize` to stronger models raised the grade from 5 to 6 and removed most priority inversions.
- **Source checking in 20261024_174514 and 20261024_183024.** A `verify_sources` node on `gpt-5-mini` used 1203k tokens. It could only remove citations, and it flagged correct ones. I replaced it with `document_pass`, which reads each document and can also add sources.
- **Baseline.** I asked `gpt-5.6-luna` once for the whole document. It took 61 s and 43k tokens and got grade 6, with the best stakeholder questions of all runs. It had only 51 requirements, 44 of them High, and it missed 8 of the 12 checked items. The baseline showed me that the network lacked a step that sees the whole corpus at once.
- **Hybrid, from 20261024_184705_hybrid on.** I put one whole-corpus `consolidate` call in place of `merge_dedup` and the coverage pass, and kept per-document extraction as a checklist. Then I banned pointer requirements such as "the reports listed in Appendix A", added source priority notes for explicit markers, and tidied the prompts. The tidy-up in `20261024_195350_hybrid` lost three checked items, and I fixed that before the final run.

Results

	hybrid	network	baseline
Requirements	136	126	51
High / Medium / Low	21 / 90 / 25	31 / 74 / 21	44 / 7 / 0
Blacklist	6	0	2
Stakeholder questions	6	0	5
Open review issues	7	4	0
Possibly missed	9	6	0
Review rounds	5 of 5	5 of 5	–
Wall time	744 s	633 s	61 s
Tokens	624,485	821,815	42,744

The `network` document has no stakeholder questions because only `consolidate` writes them. Its blacklist ended empty.

Hybrid against the final network. Opus 5.5 rated the final network run 7/10, half a point below the hybrid. Neither document misses any of the 12 checked items, the network has 2 wrong sources and the hybrid 1, and the network has the most balanced priorities of any run. Of its 25 sampled requirements 17 were accurate and 8 partly accurate, against 24 accurate in the hybrid. Several scope bullets from D17 survive as requirements, three of them at High, and an untestable "tested and proven" item stays High because no step moved it to the blacklist. One report is split in two, retention is split five ways, and the conflict about checkout at a transfer destination is decided the wrong way without being flagged. According to Opus 5.5 the gap comes from the whole-corpus `consolidate` call: it writes most of the blacklist and all stakeholder questions, and it sees both sides of a conflict.

Hybrid against the baseline. The single `gpt-5.6-luna` call produced 51 requirements, and the hybrid graph produced 136. The baseline missed 8 of the 12 checked items and only partly covered 2 more, while the hybrid document has all 12. Of 25 sampled requirements the baseline had 4 with a wrong source and the hybrid 1. The baseline does better on duplicates, about 4 items needing merge against about 20, and it is much cheaper: 61 s and 43k tokens against 744 s and 624k. Opus 5.5 rated the baseline 6/10 and the hybrid 7.5/10. Most of that difference comes from completeness.

Grading. To judge the documents I had Claude Opus 5.5 act as a requirements engineer. For each run it sampled 25 requirements across High, Medium and Low and checked each against the source documents. It also checked 12 items that some development runs had missed, counted items that need merging, and gave a grade out of 10. One model and 25 items per run make these numbers rough.

Run	Grade	Accurate	Wrong sources	Missing of 12	Needing merge
`20261024_165734`	5	11	10	12	~25
`20261024_172523`	6	21	9	3	~45
`20261024_174514`	6.5	22	7	3	~35
`20261024_183024`	6.5	22	6	5, 2 partly	~40
baseline	6	24	4	8, 2 partly	~4
`20261024_184705_hybrid`	7	23	5	3, 3 partly	~20
`20261024_191559_hybrid`	7.5	23	2	0	~15
`20261024_194017_hybrid`	7.5	23	2	0	~15
`20261024_202345_hybrid`	7.5	24	1	0	~20
`20261024_203609_network`	7	17, 8 partly	2	0	~25

Accurate and wrong sources count out of the 25 sampled requirements, and "partly" under Accurate counts requirements that are only partly right. Missing of 12 counts checked items that are absent, and "partly" those only partly covered. Needing merge is the Opus 5.5 estimate of near-duplicate items in the whole document.

Two requirements from the hybrid document:

R001 Current Last-Copy Reports, High, sources D06, D10, D14, D15
The system must provide reports identifying Items that are the only copy within a selected scope, including an individual Branch and the entire Consortium.
D06 and D15 explicitly call the last-copy report critical, and staff across the consortium depend on identifying the sole remaining copy at branch and consortium scopes.

R043 Filter Patron Reports, Medium, sources D08, D10
The system must allow patron queries and reports to be limited by age ranges calculated from Patrons' birth dates, ZIP code, county of residence, home Branch, patron type, and preferred language. Users must be able to configure the age brackets used for filtering, including user-defined brackets such as 0–5, 6–10, and 11–15, and combine these filters with other conditions using AND and OR logic.
D08 and D10 request configurable patron filters that support program planning, while the source's emphasis on importance does not make this capability necessary for system operation.

Two of its stakeholder questions:
- Should checkout at a transfer destination trigger automatic reversion, or should checked-out items remain there while unused items revert? (D04)
- What threshold should be used for the delayed in-transit report beyond the suggested configurable default of seven days? (D05)

Limitations.
- The grades come from Opus 5.5 reading 25 items per run. They only show a direction.
- Close versions of the code give different results. `20261024_195350_hybrid` lost three checked items that the hybrid runs before and after it kept.
- About 20 near-duplicates remain in the hybrid document.
- The loop almost always hits the round cap. Only `20261024_194017_hybrid` ended with the reviewer accepting on its own.
- The attribution pass costs 226k tokens, about 36% of the hybrid run, and runs without it already had most sampled sources right.

Conclusion

Per-document extraction as a checklist plus one whole-corpus consolidation call gave the best documents. Changing the list only through grouped delete-and-insert kept items from disappearing. The granularity guard, built on the origin tags, stopped the split and merge flip-flopping, and the full trace is how I found where the fine-grained items came from.

Three things failed. One call that rewrote the whole list lost items. Checking sources with a small model removed correct citations. The review loop converged only once in 20 runs, because the reviewer almost always finds something, so five rounds mostly polish and sometimes undo.

Next I would stop the loop when a round changes few requirements, add a deterministic near-duplicate check with embeddings before `review`, and run each configuration several times to measure the variance before trusting a difference.

AI assistance

The network code and this report were co-written with Claude Fable 5.1 and Claude Opus 5.5.